

Homework 03

Building Graph Representation

Graph Machine Learning — Session 06 (13A & 13B)

The Building Model

The building was modelled in Rhinoceros 3D and exported as four separate OBJ files, one for each spatial category: ground slab, structural columns, office volumes, and service core. Each file corresponds to a distinct node type in the graph and is loaded independently in the notebook.

Part 1 — Constructing the Graph (13A)

Loading and Tagging Geometry

Each OBJ file was imported using `Topology.ByOBJPath` and its faces were consolidated into closed volumetric cells via `Topology.SelfMerge`. A semantic tag — ground, column, office, or core — was then attached to each cell along with a display colour, stored as a dictionary on a selector point placed inside the volume.

Assembling the CellComplex

All tagged cells were merged into a single `CellComplex`. Wherever two spaces share a wall, that surface becomes an internal face that encodes the adjacency — each cell implicitly knows its spatial neighbours through shared boundaries.

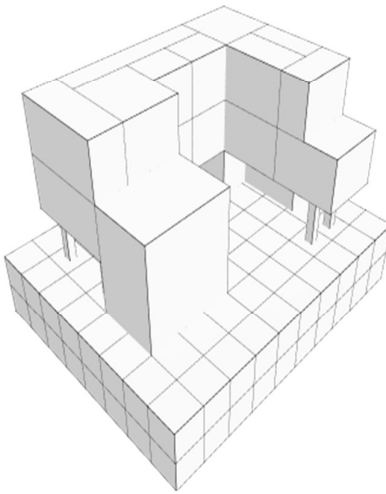


Figure 1. CellComplex — merged volumetric model

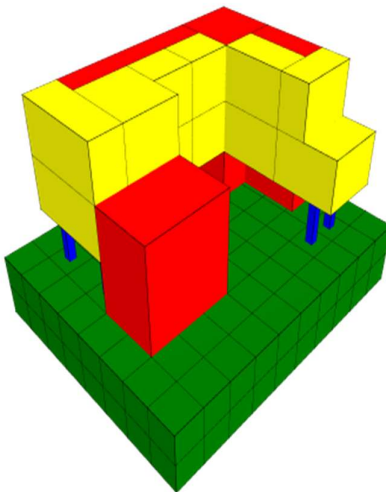


Figure 2. CellComplex with type-based colouring (ground, columns, offices, core)

Propagating Labels

The selector dictionaries were transferred onto the unified cells using `Topology.TransferDictionariesBySelectors`, ensuring that every cell in the CellComplex retains its type label after the merge operation.

Converting to a Graph

Graph.ByTopology converted the CellComplex into an attributed graph: each cell becomes a node and each shared internal face becomes an edge. The resulting graph contains 187 nodes and 870 directed edges. Node types were one-hot encoded as four-dimensional feature vectors:

```

ground → [1, 0, 0, 0]    (160 nodes)
column → [0, 1, 0, 0]    (12 nodes)
office → [0, 0, 1, 0]    (8 nodes)
core   → [0, 0, 0, 1]    (7 nodes)

```

The feature matrix was exported to CSV as feature_00 through feature_03, alongside node coordinates, edge lists, and the manually assigned graph label.

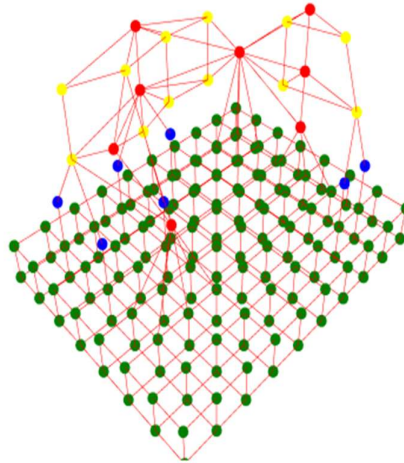


Figure 3. Adjacency graph — nodes coloured by type, edges represent shared faces

Part 2 — Predicting the Label (13B)

The CSV dataset was loaded into a PyTorch Geometric data object and passed to the pre-trained model `pyg_model.pt` via the `Predict()` function. The model performs graph-level classification into one of five building-ground relationship categories:

- Separation — the building is raised on pilotis with no direct ground contact
- Separation with Plinth — raised volume, but a solid base mediates the transition
- Adherence — floor planes sit flush on the ground without an intermediary
- Adherence with Plinth — grounded, but a podium extends and anchors the footprint
- Interlock — the ground plane penetrates into the building volume, blurring the boundary

The output table compares the manually assigned label against the model's prediction and reports a confidence score for each graph.

Actual Value	Predicted Value	Actual Label	Predicted Label	Confidence
3	2	Adherence with Plinth	Adherence	0.89

Table 1. Prediction results from `pyg_model.pt`

Reflection — Adherence with Plinth vs. Adherence

The manually assigned label was 3 (Adherence with Plinth), while the model predicted 2 (Adherence) with a confidence of 0.89. The discrepancy is meaningful and worth examining.

In the model, the ground slab is represented as a single node connecting to all 12 column nodes and to the office and core volumes. This topology — one highly-connected base node whose edges reach across multiple spatial types — is characteristic of a plinth: a continuous platform that mediates between earth and the building above it. That structural reading justified the assigned label of Adherence with Plinth.

However, the GNN classified the graph as plain Adherence. At a confidence of 0.89, this is not an ambiguous result. One likely explanation is that the ground node, despite its high degree, does not present the spatial configuration the model associates with a plinth. In training examples of Adherence with Plinth, the podium probably appears as a distinct volumetric element with its own geometry and adjacency pattern, rather than as a flat slab whose thickness is absorbed into the node feature. From the model's perspective, the building simply sits on the ground — the slab reads as the ground itself, not as an elevated base.

This gap between the manually assigned label and the model's prediction highlights an important limitation of graph-level classification: topological connectivity encodes neighbourhood relationships, but it does not directly capture geometric properties such as slab thickness, elevation, or volumetric prominence. A richer feature set — including height data, bounding-box dimensions, or surface area ratios — might allow the model to distinguish a thick, elevated plinth from a flush floor slab more reliably.